

Documentation Technique — Application Mobile Expo Go (Frontend)

Projet PokéCard Masters — BTS SIO option SLAM

1. Présentation du projet

PokéCard Masters est une application mobile de collection de cartes Pokémon TCG. Le frontend est une application **React Native** développée avec **Expo SDK 54**, qui communique avec l'API Django via des requêtes HTTP authentifiées par JWT. L'application permet à un utilisateur de s'inscrire, se connecter, consulter le catalogue de cartes, gérer sa collection et ouvrir des boosters virtuels.

L'architecture repose sur le paradigme **file-based routing** d'Expo Router, avec une gestion d'état d'authentification centralisée via un **Context React** et un stockage sécurisé du token JWT sur l'appareil.

2. Cahier des charges

2.1 Contexte

L'application mobile est le point d'entrée principal de l'utilisateur final. Elle doit fonctionner sur iOS et Android, offrir une expérience fluide et sécurisée, et s'interfacer proprement avec l'API backend.

2.2 Besoins fonctionnels

ID	Besoin	Priorité
F01	Écran d'inscription (email, mot de passe, nom)	Haute
F02	Écran de connexion (email/mot de passe)	Haute
F03	Connexion via Google OAuth2	Haute
F04	Affichage du catalogue de cartes	Haute

ID	Besoin	Priorité
F05	Affichage de la collection personnelle	Haute
F06	Ouverture de booster avec animation/retour visuel	Moyenne
F07	Navigation par onglets et tiroir latéral (drawer)	Haute
F08	Déconnexion de l'utilisateur	Haute
F09	Filtrage des cartes par type	Basse

2.3 Besoins non fonctionnels

- **Sécurité** : token JWT stocké dans `expo-secure-store` (chiffré par l'OS)
- **Compatibilité** : iOS et Android via React Native
- **Performance** : images chargées via `expo-image` (cache optimisé)
- **Maintenabilité** : TypeScript strict, architecture par dossiers fonctionnels

3. Stack technique

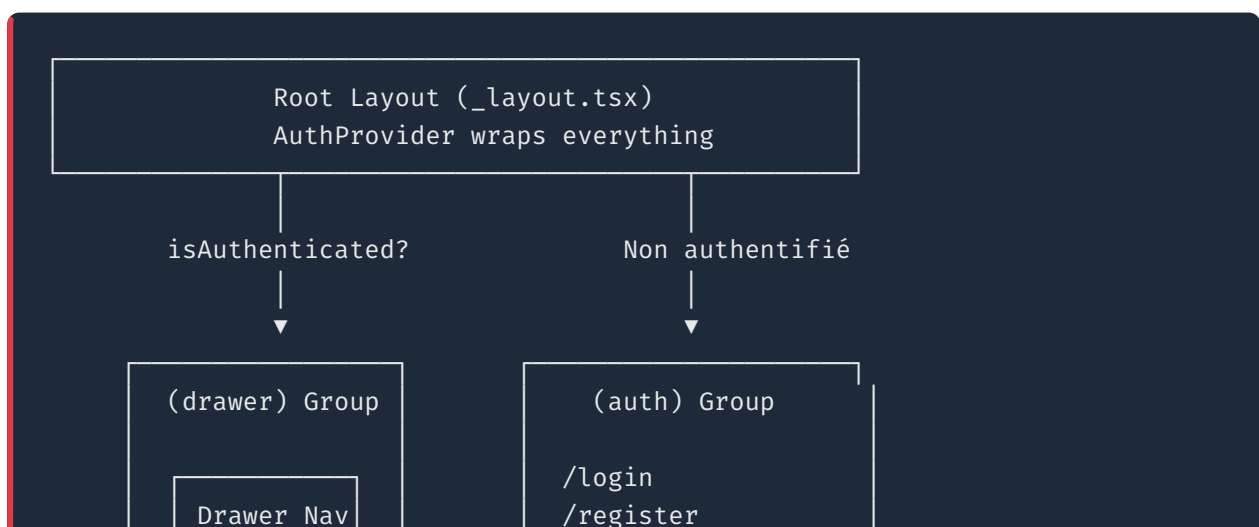
Composant	Technologie	Version
Langage	TypeScript	~5.9.2
Framework UI	React Native	0.81.5
Plateforme	Expo	SDK 54
Routing	Expo Router	~6.0.23
Navigation	React Navigation (Bottom Tabs + Drawer)	7.x
Auth OAuth2	expo-auth-session + expo-web-browser	—
Stockage sécurisé	expo-secure-store	~15.0.8
Images	expo-image	~3.0.11
Animations	react-native-reanimated	~4.1.1
Gradients	expo-linear-gradient	~15.0.8
Polices	@expo-google-fonts/press-start-2p	—

4. Architecture du projet

```
app_expo/app-pokemasters/
├── app/
│   ├── _layout.tsx           # Root layout (AuthProvider)
│   ├── (auth)/               # Groupe de routes non authentifiées
│   │   ├── login.tsx         # Écran de connexion
│   │   └── register.tsx      # Écran d'inscription
│   └── (drawer)/              # Groupe de routes authentifiées (drawer)
│       ├── _layout.tsx       # Layout drawer + tabs
│       ├── index.tsx         # Accueil / Catalogue
│       ├── collection.tsx     # Collection personnelle
│       └── booster.tsx        # Ouverture de booster
├── context/
│   └── AuthContext.tsx       # Contexte d'authentification global
├── services/
│   └── api.ts                 # Fonctions fetch vers l'API backend
├── config/
│   └── auth.ts                # Configuration OAuth2 Google
├── components/                # Composants réutilisables
├── assets/                    # Images, polices
├── package.json
└── tsconfig.json
```

5. Flux de navigation

Expo Router utilise un système de **routing basé sur les fichiers** (file-based routing), similaire à Next.js. La structure des dossiers détermine directement les routes accessibles.





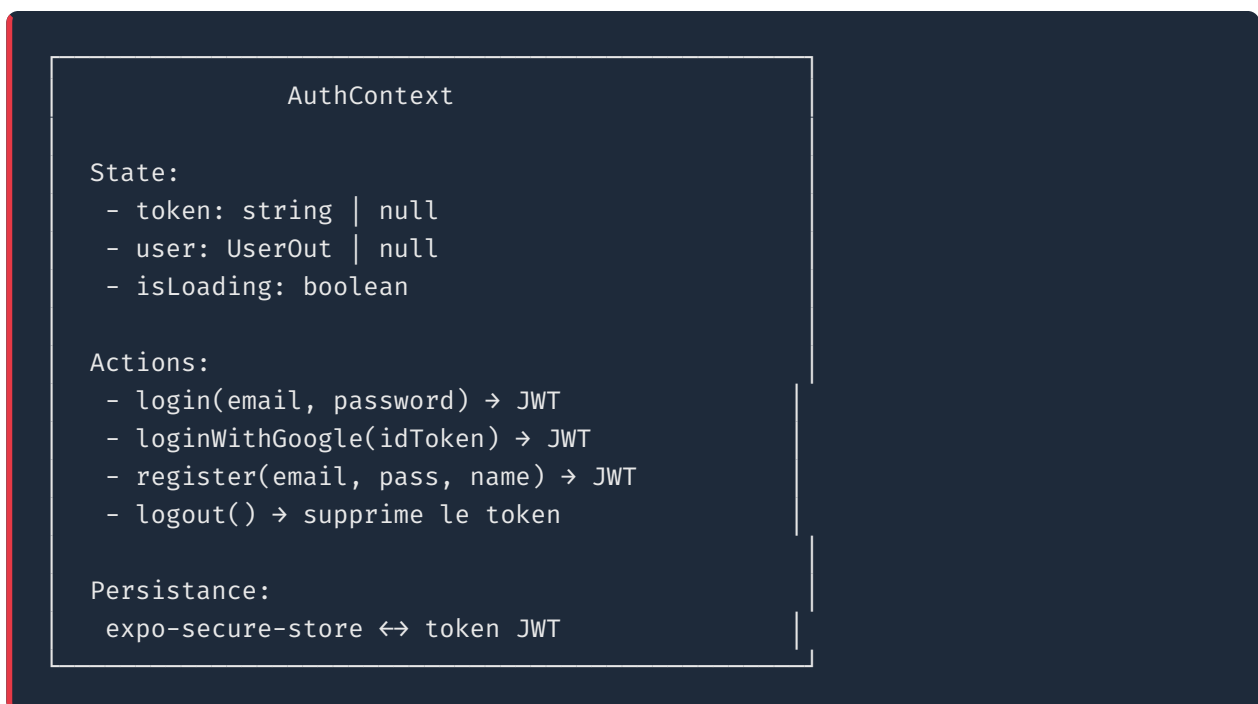
Groupes de routes

- **(auth)** : accessible sans authentification (login, register)
- **(drawer)** : accessible uniquement si un token JWT valide est présent dans `expo-secure-store`

6. Gestion de l'authentification (AuthContext)

6.1 Principe

L' `AuthContext` est un **Context React** qui encapsule toute la logique d'authentification et la rend disponible à tous les composants de l'application via le hook `useAuth()`.



6.2 Flux de connexion Google OAuth2

```
Utilisateur
|
▼ Appui sur "Connexion avec Google"
expo-auth-session (PKCE)
|
▼ Ouverture navigateur système (expo-web-browser)
Google OAuth2 Consent Screen
|
▼ Retour : authorization_code
expo-auth-session échange → ID Token Google
|
▼ POST /api/me avec ID Token en Bearer
Django API vérifie signature Google
|
▼ Création ou récupération de l'utilisateur en BDD
|
▼ Réponse : { id, name, email }
AuthContext stocke le token dans expo-secure-store
|
▼ Redirection vers (drawer)
```

6.3 Persistance du token

Le token JWT est stocké via `expo-secure-store`, qui utilise les mécanismes de sécurité natifs de l'OS (Keychain sur iOS, EncryptedSharedPreferences sur Android). Au démarrage de l'application, l'AuthContext lit ce token pour déterminer si l'utilisateur est déjà connecté.

7. Couche service (API)

Le fichier `services/api.ts` centralise toutes les communications avec le backend. Il expose des fonctions typées qui :

1. Récupèrent le token JWT depuis l'AuthContext
2. Construisent la requête HTTP avec le header `Authorization: Bearer <token>`
3. Retournent les données typées ou lancent une erreur

```
// Exemple simplifié
async function fetchCards(token: string): Promise<CardsOut[]> {
  const response = await fetch(`${API_BASE_URL}/card`, {
    headers: {
```

```

    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});
if (!response.ok) throw new Error("Erreur réseau");
return response.json();
}

```

Cette centralisation présente deux avantages : les écrans ne contiennent aucune logique HTTP, et les URLs backend sont définies en un seul endroit.

8. Composants et écrans principaux

8.1 Écrans d'authentification — (auth)/

Écran	Fichier	Description
Connexion	<code>login.tsx</code>	Formulaire email/password + bouton Google
Inscription	<code>register.tsx</code>	Formulaire email/password/nom

Les deux écrans utilisent le hook `useAuth()` pour appeler les actions de l'AuthContext.

8.2 Écrans principaux — (drawer)/

Écran	Description
Catalogue	Affiche toutes les cartes via <code>GET /card</code> . Les cartes sont affichées en grille avec leur image (expo-image), nom et type.
Collection	Affiche les cartes de l'utilisateur connecté via <code>GET /player/card</code> , avec la quantité de chaque carte.
Booster	Permet d'ouvrir un booster via <code>POST /booster/open</code> . Affiche les 10 cartes tirées avec leur rareté.

8.3 Navigation

L'application utilise deux niveaux de navigation imbriqués : - **Drawer** (tiroir latéral) : menu principal de navigation, accessible via un swipe depuis la gauche - **Bottom Tabs** : navigation rapide entre les sections principales

9. Typage TypeScript

Les types des réponses API sont définis dans le dossier `services/` ou un fichier de types dédié, en miroir des schemas Django Ninja :

```
// Types miroirs des schemas backend
type CardsOut = {
  id: number;
  name: string;
  image: string;
  types: string | null;
};

type UserOut = {
  id: number;
  name: string;
  email: string;
};

type PlayerCardSchema = {
  id: number;
  name: string;
  image: string;
  category: string | null;
  rarity: string | null;
  illustrator: string | null;
  quantity: number;
};

type BoosterCardOut = {
  name: string;
  card_id: string;
  image: string;
  category: string;
  rarity: string;
  illustrator: string;
};
```

10. Installation et démarrage

Prérequis

- Node.js (LTS recommandé)
- Application **Expo Go** sur iOS ou Android, OU un simulateur

- Backend Django démarré sur `http://localhost:8000`

Étapes

```
# 1. Naviguer dans le dossier frontend
cd app_expo/app-pokemasters

# 2. Installer les dépendances Node
npm install

# 3. Démarrer le serveur de développement Expo
npx expo start

# Pour iOS simulator
npx expo start --ios

# Pour Android emulator
npx expo start --android
```

Configuration

S'assurer que l'URL de l'API backend dans `services/api.ts` (ou `config/`) pointe vers : - `http://localhost:8000` en développement local - L'IP de la machine sur le réseau local si test sur appareil physique (ex : `http://192.168.1.x:8000`)

11. Commandes utiles

```
# Lancer le linter ESLint
npx expo lint

# Nettoyer le projet (reset)
node ./scripts/reset-project.js

# Vérifier les types TypeScript
npx tsc --noEmit
```


12. Dépendances clés — Justification des choix

Librairie	Rôle	Justification
<code>expo-auth-session</code>	OAuth2 / PKCE	Gère le flux OAuth2 de manière sécurisée sur mobile
<code>expo-secure-store</code>	Stockage JWT	Chiffrement natif OS, plus sûr qu'AsyncStorage
<code>expo-image</code>	Affichage images	Cache performant, meilleur que <code><Image></code> natif
<code>react-native-reanimated</code>	Animations	Animations 60fps sur le thread natif
<code>@react-navigation/drawer</code>	Menu latéral	Navigation par tiroir latéral, standard mobile
<code>expo-linear-gradient</code>	Design	Dégradés visuels pour le thème Pokémon
<code>@expo-google-fonts/press-start-2p</code>	Police	Police pixel-art pour l'ambiance rétro Pokémon

13. Glossaire

Terme	Définition
Expo Router	Système de routing basé sur les fichiers pour Expo, inspiré de Next.js
Context React	Mécanisme de partage d'état global sans prop drilling
expo-secure-store	Module Expo stockant des données chiffrées dans le Keychain (iOS) ou EncryptedSharedPreferences (Android)
PKCE	Proof Key for Code Exchange — extension OAuth2 sécurisant les flux publics (applications mobiles)
File-based routing	Système où la structure des fichiers détermine les routes de navigation
Bottom Tabs	Barre de navigation en bas de l'écran avec plusieurs onglets

Terme	Définition
Drawer	Menu de navigation latéral accessible par swipe ou bouton
JWT	JSON Web Token — token signé transportant l'identité de l'utilisateur
AsyncStorage	Stockage clé/valeur non chiffré de React Native (non utilisé ici pour le token)
